Covercrypt

An efficient ABE-like encryption scheme.

Contents

1 Introduction **2**

2 Preliminaries **2**
- 2.1 Non-Interactive Key Exchange (NIKE) . . . 2
- 2.2 Key-Encapsulation Mechanism (KEM) . . . 3
- 2.3 KEM with Access Control (KEM-AC) . . . 3

3 Subset-Cover KEM-AC (SC-KEM-AC) **4**
- 3.1 KEM-AC specification . . . 4
- 3.2 Key and encapsulation life-cycle management . . . 4
- 3.3 **TODO** Hybridization à la carte . . . 5
- 3.4 Live access-structure modifications . . . 6
- 3.5 SC-KEM-AC reference implementation . . . 6
  - 3.5.1 Key format . . . 6
  - 3.5.2 SC-KEM-AC primitives . . . 6
  - 3.5.3 Key-rotation primitives . . . 9
  - 3.5.4 Live-universe-modification primitive . . . 11

4 Boolean KEM-AC **11**
- 4.1 Syntax and semantics . . . 11
- 4.2 KEM-AC . . . 12
  - 4.2.1 Examples . . . 12
- 4.3 **TODO** Live universe modifications . . . 12
- 4.4 **TODO** Hybridization à la carte . . . 13

5 TODO Optional key tracing **13**

6 TODO Cooperating Covercrypt authorities **13**

1 Introduction

In large organizations, pieces of data created by one member are commonly shared to other members. However, some pieces of data are sensitive and must only be shared to a specific subset of members. The process of accessing a piece of data therefore becomes a three-step procedure requiring members to: (1) authenticate themselves to some trusted identity management system, (2) request the authorization to access this data from some trusted authorization management system and (3) request this data from the storage system which first verifies the validity of the identity claim delivered by the identity management system, the validity of the authorization claim delivered by the authorization management system, and the compatibility of those claims.

However, authorization rules may be complex which may lead to operator errors or bugs in the authorization and storage systems. Moreover actors may misbehave, which may question our trust in the authorization given by those systems. Hopefully, there are ways to raise the need for trust in those systems. One of them consists in cryptographically enforcing authorization by encrypting data under a key owned by the members of the targeted only. That way, the only trusted parties are the targeted members and the key distribution, which can be performed in a centralized Key Management System (KMS). But if authorization rules are complex, key distribution may also become complex and hence victim of the same kind of errors.

Attribute-Based Encryption (ABE) is a tool that has been extensively studied to solve exactly this problem. However, known ABE schemes are not very efficient and not post-quantum-secure which hinders their adoption. Covercrypt solves the same problem while relaxing some formal constraints in order to achieve efficiency in most practical use-cases while offering both CCA and post-quantum security, attribute-hiding ciphertexts and key tracing.

Covercrypt assumes the existence of a centralized master authority holding a master secret key with which all other keys are created: the master public key is used to encrypt data under its access policy while user secret keys are also associated to access policies and can decrypt all Covercrypt ciphertexts with a compatible access policy. Covercrypt follows the KEM-DEM approach in which data is symmetrically encrypted under a key which encapsulation is appended to the symmetric ciphertext. More precisely, Covercrypt is a actually Key-Encapsulation Mechanism with Access Control (KEM-AC) – as defined in 2.3 – that can be combined with a Data-Encryption Mechanism (DEM) to obtain a Public Key Encryption (PKE) scheme. In the rest of this document, we therefore prefer the term Covercrypt *encapsulation* to the term Covercrypt ciphertext. Moreover, when necessary, we use the terms *user policy* and *encapsulation policy* to denote an access policy associated to a user secret and a Covercrypt encapsulation respectively, and use the term policy the context makes it clear whether we refer to a user or encapsulation policy.

2 Preliminaries

2.1 Non-Interactive Key Exchange (NIKE)

- Note taken on *[2026-07-21 Tue 19:32]*
  The key-homomorphic definition from the ETSI document is insufficient to allow for an easy computation of $\beta = (s - \alpha \cdot s_1) * s_2^{-1}$. For this, we really need the notion of field since secret keys must be invertible with respect to the multiplicative law. Then, we need the notion that there exists a group generator $\mathfrak{g}$ for the public keys such that for any pair of secret and public keys (sk, pk), $pk = sk \cdot \mathfrak{g}$.

Definition 2.1. A NIKE is a cryptographic scheme allowing to privately establish a session key between two parties in a non-interactive fashion using the following two primitives:

keygen: $\perp \to \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm returns a new pair of secret and public keys (sk, pk).

session-key: $\mathcal{K}_P \times \mathcal{K}_S \to \mathcal{K}$

Upon receiving a public key $pk \in \mathcal{K}_P$ and a secret key $sk \in \mathcal{K}_S$, returns the associated session key $K \in \mathcal{K}$ such that for any given pairs of keys (sk_1, pk_1) and (sk_2, pk_2), session-key$(pk_1, sk_2) =$ session-key(pk_2, sk_1).

Additionally, a NIKE is said to be *key-homomorphic* if its public keys form an Abelian group which scalars are its secret keys.

Remark. One instantiation of a key-homomorphic NIKE is based on an elliptic-curve group G with prime order p and group generator $\mathfrak{g}$ and a hash function $\mathcal{H} : G \to \{0,1\}^\lambda$:

keygen: returns $sk \xleftarrow{\$} \mathcal{U}(0, p-1)$ and $pk \leftarrow sk \cdot \mathfrak{g}$.

session-key: upon receiving $pk = sk \cdot \mathfrak{g}$ and sk', returns $K \leftarrow \mathcal{H}(sk \cdot sk' \cdot \mathfrak{g})$.

2.2 Key-Encapsulation Mechanism (KEM)

Definition 2.2. A KEM is a cryptographic scheme associated to the key spaces $\mathcal{K}_D$, $\mathcal{K}_E$ and $\mathcal{K}$, and encapsulation space $\mathcal{E}$, that exposes the following primitives:

keygen: $\bot \to \mathcal{K}_D \times \mathcal{K}_E$

This randomized algorithm returns a new pair of decapsulation and encapsulation keys (dk, ek).

encaps: $\mathcal{K}_P \to \mathcal{K} \times \mathcal{E}$

This randomized algorithm takes as input an encapsulation key $ek \in \mathcal{K}_E$ and returns a new session key $K \in \mathcal{K}$ and its encapsulation $\mathsf{enc} \in \mathcal{E}$.

decaps: $\mathcal{K}_S \times \mathcal{E} \to \mathcal{K}$ or $\bot$

This algorithm takes as input a decapsulation key $dk \in \mathcal{K}_D$ and an encapsulation $\mathsf{enc} \in \mathcal{E}$, and returns the encapsulated session key K if and only if the encapsulation was created for the encapsulation key associated to dk.

2.3 KEM with Access Control (KEM-AC)

A Key-Encapsulation Mechanism with Access Control (KEM-AC) allows encapsulating a secret with respect to an *encapsulation policy*, such that user secret key can open it if an only if its associated *user policy* is compatible with the encapsulation policy with respect to some binary relation over the policy space.

Definition 2.3 (Key-Encapsulation Mechanism with Access Control)**.** A KEM-AC is defined over a master-secret-key space $\mathcal{K}_S$, master-public-key space $\mathcal{K}_P$ and a user-secret-key space $\mathcal{K}_U$, a session-key space $\mathcal{K}$ and an encapsulation space $\mathcal{E}$, and exposes the following operations:

setup: $\mathbb{P}(\mathcal{P}_U \times \mathcal{P}_E) \to \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a binary relation R over policies, and returns a master secret key msk_R and a master public key mpk_R.

keygen: $\mathcal{K}_S \times \mathcal{P}_U \to \mathcal{K}_U$

This randomized algorithm takes as input a master secret key $\mathsf{msk}_R \in \mathcal{K}_S$ under a binary relation $R \subset \mathcal{P}_U \times \mathcal{P}_E$ and a policy $p_u \in \mathcal{P}_U$, and returns a user secret key $\mathsf{usk}_{p_u,R}$.

encaps: $\mathcal{K}_P \times \mathcal{P}_E \to \mathcal{K} \times \mathcal{E}$

This randomized algorithm takes as input a master public key $mpk_R \in \mathcal{K}_P$ under a binary relation $R \subset \mathcal{P}_U \times \mathcal{P}_E$ and an encapsulation policy $p_e \in \mathcal{P}_E$, and returns both a session key $k \in \mathcal{K}$ and its encapsulation $\mathsf{enc}_{p_e,R} \in \mathcal{E}$.

decaps: $\mathcal{K}_U \times \mathcal{E} \to \mathcal{K}$ or $\bot$

This algorithm takes as input a user secret key $\mathsf{usk}_{p_u,R_u} \in \mathcal{K}_U$ associated to an access policy $p_u \in \mathcal{P}_U$ under a binary relation $R_u \subset \mathcal{P}_U \times \mathcal{P}_E$, and an encapsulation $\mathsf{enc}_{p_e,R_e} \in \mathcal{E}$ associated to a policy $p_e \in \mathcal{P}_E$ under a binary relation $R_e \subset \mathcal{P}_U \times \mathcal{P}_E$. It returns the encapsulated session key $k \in \mathcal{K}$ if $(p_u, p_e) \in R_e$, and $\bot$ otherwise.

3 Subset-Cover KEM-AC (SC-KEM-AC)

3.1 KEM-AC specification

- Note taken on *[2026-10-13 Tue 14:55]*
  TODO: List security claims somewhere.

Covercrypt is a subset-cover algorithm: it implements a KEM-AC with respect to an access structure S defining a set of rights $\mathcal{U} = \{r_i\}_i$ called *universe*, which policy space is $\mathbb{P}(\mathcal{U})$ and which binary relation is the intersection test: $R_{\mathcal{U}} = \{(p_u, p_e) \in \mathbb{P}(\mathcal{U})^2 : p_u \cap p_e \neq \emptyset\}$. The complete reference implementation is given in Section 3.5 (note that the format of the keys is dictated by the additional requirements listed in the following sections).

3.2 Key and encapsulation life-cycle management

Just like any other keys, Covercrypt keys can be compromised and lost. There is therefore the need to renew them. However, in order to preserve the ability to open older encapsulations with renewed keys, it is desirable for the key renewal to be different from setting up a new instance. The renewal procedure – called *key rotation* – is described in the present section.

Recall that each user secret key usk_p is associated to a policy p. Upon compromise of user secret key, a key rotation with respect to its entire policy must be performed. Due to the particular structure of Covercrypt keys, this process implies: (1) the rotation of all associated secrets in the master secret key, (2) the propagation of this rotation to the master public key to prevent future encapsulations from being using compromised secrets, (3) the propagation of this rotation to all user secret keys which policy intersects the ones of the compromised key to allow legitimate users opening the new encapsulations generated with respect to the new secrets, and (4) optionally the refresh of all encapsulations which associated policy intersects the one of the compromised key to prevent an eventual attacker from opening them if it still hasn't accessed them. The entire key-rotation process is depicted in Figure 1, and leverages the following primitives:

usk-policy: $\mathcal{K}_U \to \mathbb{P}(\mathcal{U})$

This function takes as input a Covercrypt user secret key $usk_p \in \mathcal{K}_U$ and returns its associated policy $p \subset \mathcal{U}$.

rotate: $\mathcal{K}_S \times \mathbb{P}(\mathcal{U}) \to \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $\mathsf{msk}_{\mathcal{U}} \in \mathcal{K}_S$ and an access policy $p^* \subset \mathcal{U}$, refreshes every secrets in this given master secret key that are associated to a right in this policy (*i.e.* each secret associated to a right in $s \cap p^*$), and returns the refreshed master secret key along with its associated master public key.

usk-refresh: $\mathcal{K}_S \times \mathcal{K}_U \to \mathcal{K}_U$ or $\perp$

This algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ and a user secret key $usk_p \in \mathcal{K}_U$ associated to a policy $p \subset \mathcal{U}$. If the integrity of the given user secret key can be verified by the master secret key, it returns a new user secret key usk'_p associated to the same access policy containing all secrets that are contained in the given user secret key, plus new secrets for each rights in p for which newer secrets exists in the master secret key. If the integrity of the given user secret key cannot be verified, $\perp$ is returned.

enc-refresh: $\mathcal{K}_S \times \mathcal{K}_P \times \mathcal{E} \to \mathcal{E}$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$, a master public key $mpk \in \mathcal{K}_S$ and an encapsulation $enc_p \in \mathcal{E}$ associated to a policy $p \subset \mathcal{U}$. If the integrity of the master public key can be verified, that its revision number matches that of the master secret key, and that the encapsulation can be open, it returns a new encapsulation enc'_p associated to the same access policy. Otherwise $\perp$ is returned.

Remark. Checking the integrity of the user secret key (resp. encapsulation key) upon refresh is required to prevent right escalation (resp. impersonification) via modification of this key, while encapsulation policies cannot be modified via modification of the encapsulation due to the CCA security guaranteed by Covercrypt encapsulations.

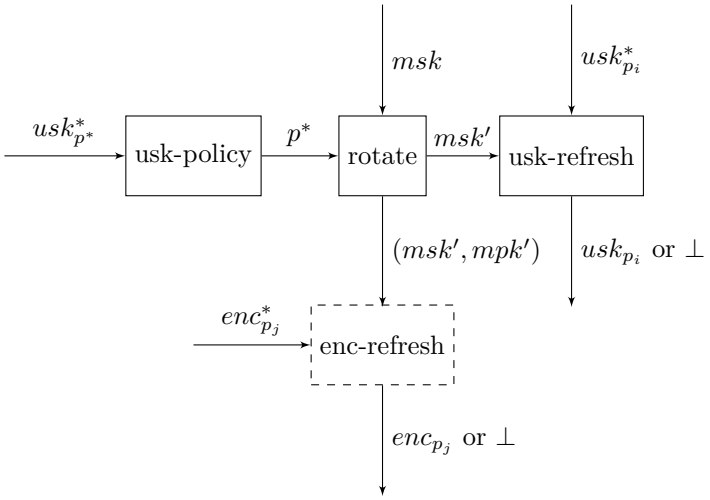


Figure 1: User-secret-key rotation flow, where untrusted objects are marked with an asterix.

Additionally, to prevent impersonification attacks upon encapsulation, users can always verify the integrity of the master public key using user secret keys, and since this is necessary to prevent right-escalation attacks upon user-secret-key refresh, we also provide a way to verify the integrity of user secret keys using the master secret key:

mpk-verify: $\mathcal{K}_U \times \mathcal{K}_P \to \mathcal{B}$

Returns whether the integrity of the given master public key $mpk \in \mathcal{K}_P$ can be verified using the given user secret key $usk_p \in \mathcal{K}_U$.

usk-verify: $\mathcal{K}_S \times \mathcal{K}_P \to \mathcal{B}$

Returns whether the integrity of the given user secret key $usk_p \in \mathcal{K}_U$ can be verified using the given master secret key $msk \in \mathcal{K}_S$.

Finally, a Covercrypt instantiation is associated to a sequence of *revisions*: the setup initializes an instance with *revision number* 0, and each execution of the key rotation procedure increments this number by one and returns the new value of the master secret key with the new revision number. Covercrypt objects can only create objects with the same revision number which can be extracted via one of the following functions:

msk-revision: $\mathcal{K}_S \to \mathbb{N}$

Retrieves the revision number from the given master secret key $\mathsf{msk} \in \mathcal{K}_S$.

mpk-revision: $\mathcal{K}_P \to \mathbb{N}$

Retrieves the revision number from the given master public key $\mathsf{mpk} \in \mathcal{K}_P$.

usk-revision: $\mathcal{K}_U \to \mathbb{N}$

Retrieves the revision number from the given master public key $\mathsf{usk} \in \mathcal{K}_U$.

enc-revision: $\mathcal{E} \to \mathbb{N}$

Retrieves the revision number from the encapsulation $\mathsf{enc} \in \mathcal{E}$.

Remark. Downgrade attacks upon encapsulation can be avoided by verifying the integrity of the master public key used, and that its revision number is no less than that of the user secret key.

3.3 TODO Hybridization à la carte

To accommodate different regulations and right sensitivities, it is possible to describe for each right the kind of encapsulation to generate, among a set of predefined encapsulation kinds.

3.4 Live access-structure modifications

Real-life access structure may evolve with time. Covercrypt reflects this by exposing primitives allowing to modify the policy universe without having to perform a new setup. In particular, additive modifications of the universe are *backward-compatible*: old user secret keys and encapsulations need not be modified in order to open new compatible encapsulations, or to be opened by new compatible user secret keys. In other words, additive universe modifications can be seen as the use of a right that was present in the universe but unused until then.

add-rights: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \to \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ under a universe $\mathcal{U}$ and a set of new rights $\{r_i\}_i \subset \mathcal{R}$, and returns new master secret and public keys under a universe $\mathcal{U}' \leftarrow \mathcal{U} \cup \{r_i\}_i$.

del-rights: $\mathcal{K}_S \times \mathcal{R} \to \mathcal{K}_S \times \mathcal{K}_P$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ under a universe $\mathcal{U}$ and a set of new rights $\{r_i\}_i \subset \mathcal{R}$, and returns new master secret and public keys with incremented revision numbers under a universe $\mathcal{U}' \leftarrow \mathcal{U} \setminus \{r_i\}_i$.

Remark. Additive universe modifications do not generate new revisions while destructive universe modifications generate new revisions since downgrade attacks can leverage master public keys prior to the removal of some rights from the universe to trick users create encapsulations for compromised secrets.

3.5 SC-KEM-AC reference implementation

In this section, we give a reference implementation for the Covercrypt SC-KEM-AC algorithm. This implementation is optimized for simplicity and may not be optimized for speed. It presupposes the existence of a key-homomorphic NIKE and KEM schemes as described in the Section 2.

3.5.1 Key format

3.5.2 SC-KEM-AC primitives

This implementation presuppose the existence of hash functions $\mathtt{G}, \mathtt{H}, \mathtt{J}, \mathtt{U}, \mathtt{T}$ which signatures are as follows:

- $G : \{0,1\}^* \to \mathcal{K}_S^{\text{nike}}$
- $T : (\mathcal{E}^{\text{kem}})^* \to \mathcal{T}_T$
- $U : (\{0,1\}^*)^* \to \mathcal{T}_U$
- $H : \mathcal{K}^{\text{nike}} \times \mathcal{K}^{kem} \times \mathcal{K}_P^{\text{nike}} \times \mathcal{K}_P^{\text{nike}} \times \mathcal{T}_T \to \{0,1\}^*$
- $J : \{0,1\}^* \times \mathcal{K}_P^{\text{nike}} \times \mathcal{K}_P^{\text{nike}} \times \mathcal{T}_U \to \{0,1\}^* \times \{0,1\}^*$

Or, alternatively:

```
module type G = sig
  val hash : bytes -> Nike.sk
end

module type T = sig
  type t

  val hash : Kem.enc list -> t
end

module type U = sig
  type t

  val hash : bytes list -> t:T.t -> t
end
```

```
module type H = sig
  val hash : Nike.key -> Kem.key -> c1:Nike.pk -> c2:Nike.pk -> T.t -> bytes
end

module type J = sig
  type key
  type tag

  val hash : secret:bytes -> c1:Nike.pk -> c2:Nike.pk -> U.t -> key * tag
end
```

Note that the output of `T` and `U` are abstract and thus different from any other type, and that the `key` output of `J` is the type of the key returned by the `encaps` and `decaps` primitives. With this in mind, the reference implementation of the primitives of Covercrypt SC-KEM-AC is as follows:

setup: $\mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$

Given the following master secret and public key types,

```
  rsk : (Nike.sk * Kem.dk) RightMM.t;
  seal : Sig.seal;
}

type enc = {
  v : nat;
  tag : tag;
  c1 : Nike.pk;
  c2 : Nike.pk;
  xenc : (Kem.enc * keylen sbytes) list;
}

let rpk_to_bytes rpk =
  RightMap.fold
    (fun right (pk, ek) bytes ->
      Right.to_bytes right :: Nike.pk_to_bytes pk :: Kem.ek_to_bytes ek
      :: bytes)
```

the setup is a randomized algorithm that takes as input a universe $\mathcal{U}$, and returns a master secret key $\mathsf{msk}_{R_\mathcal{U}}$ and a master public key $\mathsf{mpk}_{R_\mathcal{U}}$.

```
let encaps rng mpk policy =
  let pks =
    Policy.fold
      (fun right acc ->
        match RightMap.get mpk.rpk right with
        | Some (pk, ek) -> (pk, ek) :: acc
        | None -> invalid_arg "invalid policy")
      policy []
  in
  let secret = SBytes.gen rng keylen in
  let r = G.hash secret in
  let c1 = Nike.(r * mpk.h1) in
  let c2 = Nike.(r * mpk.h2) in
  let encs =
```

Note the companion procedure `get_mpk msk` given below can be used to the generate master public keys from the master secret key after the setup.

```
  let vk = Sig.get_vk msk.sk in
  let rsk =
    Policy.fold
      (fun r ->
        match RightMM.get msk.rsk r with
        | None -> invalid_arg "invalid policy"
        | Some secrets -> RightMM.add r (NonEmptyList.head secrets))
      policy msk.rsk
  in
  let v = msk.v in
  let seal = sign_usk ~sk:msk.sk ~v ~u1 ~u2 ~h1 ~h2 ~rsk in
  { v; u1; u2; h1; h2; vk; rsk; seal }
```

keygen: $\mathcal{K}_S \times \mathcal{P}_U \to \mathcal{K}_U$

```
    @@ List.map
         (fun (pk, ek) ->
           let k = Nike.session_key pk r in
           let k', enc = Kem.encaps rng ek in
           (k, k', enc))
         pks
  in
  let t = T.hash @@ List.map (fun (_, _, e) -> e) encs in
  let xenc =
    List.map
      (fun (k, k', e) ->
        let h = H.hash k k' ~c1 ~c2 t in
        let f = SBytes.xor secret h in
        (e, f))
      encs
  in
```

encaps: $\mathcal{K}_P \times \mathcal{P}_E \to \mathcal{K} \times \mathcal{E}$

```
      let u = U.hash ~t @@ List.map Pair.snd xenc in
      J.hash ~secret ~c1 ~c2 u
    in
    (key, { v = mpk.v; tag; c1; c2; xenc })

  let usk_secrets usk =
    interleave @@ List.of_seq
    @@ Seq.map NonEmptyList.to_list
    @@ RightMM.to_val_seq usk.rsk

  let try_decaps enc p h1 h2 t u sk dk e f =
    match Kem.decaps dk e with
    | None -> None
    | Some k' ->
        let k = Nike.session_key p sk in
        let secret = SBytes.xor f @@ H.hash k k' ~c1:enc.c1 ~c2:enc.c2 t in
        let key, tag = J.hash ~secret ~c1:enc.c1 ~c2:enc.c2 u in
        if tag = enc.tag then
          let r = G.hash secret in
          let c1 = Nike.(r * h1) in
          let c2 = Nike.(r * h2) in
          if enc.c1 = c1 && enc.c2 = c2 then Some key else None
        else None
```

```
let decaps usk enc =
  let p1 = Nike.(usk.u1 * enc.c1) in
  let p2 = Nike.(usk.u2 * enc.c2) in
  let p = Nike.Point.(p1 + p2) in
  let t = T.hash @@ List.map (fun (e, _) -> e) enc.xenc in
  let u = U.hash ~t @@ List.map Pair.snd enc.xenc in
  List.fold_left
    (fun key (sk, dk) ->
```

decaps: $\mathcal{K}_U \times \mathcal{E} \to \mathcal{K}$ or $\perp$

```
let p = Nike.Point.(p1 + p2) in
let t = T.hash @@ List.map (fun (e, _) -> e) enc.xenc in
let u = U.hash ~t @@ List.map Pair.snd enc.xenc in
List.fold_left
  (fun acc (r, sk, dk) ->
    List.fold_left
      (fun acc (e, f) ->
        match (try_decaps enc p h1 h2 t u sk dk e f, acc) with
        | None, acc -> acc
        | Some key, None -> Some (key, Policy.add r Policy.empty)
        | Some key, Some (key', policy) ->
            if key = key' then Some (key, Policy.add r policy)
            else invalid_arg "invalid encapsulation")
      acc enc.xenc)
```

Where `try_decaps` attempts opening a right encapsulation as follows:

```
let msk_secrets (msk : msk) =
  (* TODO: once a key in a history has opened the encapsulation, other keys
       from the same history need to attempt opening it again. *)
  interleave @@ List.of_seq
  @@ Seq.map (fun (r, secrets) ->
      List.map (fun (sk, dk) -> (r, sk, dk)) (NonEmptyList.to_list secrets))
  @@ RightMM.to_seq msk.rsk

let enc_policy msk enc =
  let p1 = Nike.(enc.c1) in
  let p2 = Nike.(Scalar.((msk.s - msk.s1) / msk.s2) * enc.c2) in
  let h1 = Nike.get_pk msk.s1 in
```

3.5.3 Key-rotation primitives

usk-policy: $\mathcal{K}_U \to \mathbb{P}(\mathcal{U})$

Extracting the policy out of a user secret key simply resorts to retrieving the rights for which it holds secrets:

```
else if msk.v != mpk.v then invalid_arg "outdated master public key"
else
```

rotate: $\mathcal{K}_S \times \mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$

```
    |> Option.map (fun (old_key, policy) ->
        let new_key, enc = encaps rng mpk policy in
        (~old_key, ~new_key, enc))

let msk_revision (msk : msk) = msk.v
let mpk_revision (mpk : mpk) = mpk.v
let usk_revision (usk : usk) = usk.v
let enc_revision (enc : enc) = enc.v
```

usk-refresh: $\mathcal{K}_S \times \mathcal{K}_U \to \mathcal{K}_U$ or $\perp$

```
let usk_policy usk =
  RightMM.fold (fun r _ -> Policy.add r) usk.rsk Policy.empty

let rotate rng (msk : msk) policy =
  let v = Int32.succ msk.v in
  let rsk =
    Policy.fold
      (fun r -> RightMM.add r (Nike.gen_sk rng, Kem.gen_dk rng))
      policy msk.rsk
  in
  { msk with v; rsk }

let usk_refresh (msk : msk) usk keep_old_secrets =
  if not (is_valid_usk msk usk) then invalid_arg "invalid user secret key"
  else
    let v = msk.v in
    let rsk =
      RightMM.fold
        (fun r secrets rsk ->
          match RightMM.get msk.rsk r with
          | None -> rsk
          | Some master_secrets ->
              if keep_old_secrets then
                let newer_secrets =
                  NonEmptyList.take_while
                    (fun s -> s != NonEmptyList.head secrets)
                    master_secrets
```

enc-refresh: $\mathcal{K}_S \times \mathcal{K}_P \times \mathcal{E} \to (\mathcal{K} \times \mathcal{E})$ or $\perp$

```
List.fold_left
  (fun acc (e, f) ->
    match (try_decaps enc p h1 h2 t u sk dk e f, acc) with
    | None, acc -> acc
    | Some key, None -> Some (key, Policy.add r Policy.empty)
    | Some key, Some (key', policy) ->
        if key = key' then Some (key, Policy.add r policy)
        else invalid_arg "invalid encapsulation")
  acc enc.xenc)
```

Where `enc_policy` allows retrieving the policy of an encapsulation along with the encapsulated key:

```
        enc.xenc key)
    None
  @@ usk_secrets usk

let msk_secrets (msk : msk) =
  (* TODO: once a key in a history has opened the encapsulation, other keys
       from the same history need to attempt opening it again. *)
  interleave @@ List.of_seq
  @@ Seq.map (fun (r, secrets) ->
      List.map (fun (sk, dk) -> (r, sk, dk)) (NonEmptyList.to_list secrets))
  @@ RightMM.to_seq msk.rsk

let enc_policy msk enc =
```

```
let p1 = Nike.(enc.c1) in
let p2 = Nike.(Scalar.((msk.s - msk.s1) / msk.s2) * enc.c2) in
let h1 = Nike.get_pk msk.s1 in
let h2 = Nike.get_pk msk.s2 in
let p = Nike.Point.(p1 + p2) in
let t = T.hash @@ List.map (fun (e, _) -> e) enc.xenc in
let u = U.hash ~t @@ List.map (fun (_, f) -> f) enc.xenc in
List.fold_left
```

3.5.4 Live-universe-modification primitive

4 Boolean KEM-AC

In order to make expressing policies more ergonomic, we introduce KEM-AC which policy space is a set of logical propositions that is closed under the two logical operators *and* ($\wedge$) and *or* ($\vee$). Then, we show how to implement this boolean KEM-AC on top of the set-based KEM-AC defined in Section 3 via compiling boolean policies to set-based policies.

4.1 Syntax and semantics

The policy language has a very simple syntax which specification is given below, and which semantics is defined thereafter.

```
<policy>    ::= <attribute> | <policy> <operator> <policy>
<operator>  ::= "&&" | "||"
<attribute> ::= "*" | <dimension>::<name>
<dimension> ::= <name>
<name>      ::= <word> \ { "&&" | "||" | <word>::<word> }
<word>      ::= char+
```

Definition 4.1 (Universe)**.** In the boolean KEM-AC setting, the universe Ω is a set of atomic logical propositions called *attributes*.

Definition 4.2 (Policies)**.** The space of policies $\mathcal{P}_\Omega$ associated to a universe of attributes Ω is the closure of Ω with respect to both logical operators *and* ($\wedge$) and *or* ($\vee$), noted $\mathcal{P}_\Omega = \langle \Omega, \wedge, \vee \rangle$. In other words, a policy is a function of sets of true attributes in Ω to the set of boolean values $\mathcal{B}$: $\mathcal{P}_\Omega \simeq \mathbb{P}(\Omega)^{\mathcal{B}}$. Given a set of true attributes $X \subset \Omega$, we therefore use $p(X)$ to denote the value taken by a policy $p \in \mathcal{P}_\Omega$ when exactly those attributes are set to true.

Remark. $\mathcal{P}_\Omega$ has two special policies, noted True and False, that are respectively always true and false.

In order to structure the set of expressible policies, the values attributes can take can be constrained by the definition of an access structure grouping logically-related attributes together by either anarchical or hierarchical dimensions.

Definition 4.3 (Dimension)**.** A dimension D of a universe of attributes Ω is a constraint on a non-empty set of attributes from Ω requiring that at least one of them be true:

$$\bigvee_{a \in D} a = \mathsf{True} \tag{dimension}$$

The set of dimensions of Ω, noted $\mathcal{D}_\Omega$, is therefore the set of its non-empty parts: $\mathcal{D}_\Omega = \mathbb{P}(\Omega)^*$.

Definition 4.4 (Hierarchy)**.** A dimension $D \in \mathcal{D}_\Omega$ defined by an access structure S is said to be a hierarchy if it is associated a bijective function $\iota \in D^{[1,|D|]}$ in S. In that case, it additionally requires that the logical implication be a strict total order over D:

$$\forall (a, a') \in D^2, a \implies a' \iff \iota(a) > \iota(a') \tag{hierarchy}$$

Definition 4.5 (Anarchy)**.** Otherwise, a dimension $D \in \mathcal{D}_\Omega$ is said to be an anarchy and additionally requires its attributes to be mutually exclusive:

$$\forall (a, a') \in D^2, (a = a') \vee (a \implies \neg a') \tag{anarchy}$$

Not all combinations of dimensions can be defined by a given access structure.

Definition 4.6 (Access Structure)**.** In the boolean KEM-AC setting, an access structure is a set of mutually disjoint dimensions:

$$\mathcal{S} = \{S \in \mathbb{P}(\mathcal{D}) \mid \forall (D_i, D_j) \in S^2, D_i \cap D_j = \emptyset\}$$

Definition 4.7 (Set of Expressible Postulates)**.** The set of expressible postulates $\mathcal{U}_S \subset \mathbb{P}(\Omega)$ described by an access structure S is the set of parts of Ω that are valid with respect to each dimension in S.

Definition 4.8 (Valid Policy)**.** A policy p is said to be valid with respect to an access structure S, noted $p \in \mathcal{V}_S$ if and only if there exists a postulate in $\mathcal{U}_S$ that validates p:

$$p \in \mathcal{V}_S \iff \exists X \in \mathcal{U}_S : p(X) = \mathsf{true}$$

4.2 KEM-AC

In the boolean KEM-AC setting, a user secret key $\mathsf{usk}_{p_u,S}$ associated to a policy $p_u \in \mathcal{P}_\mathcal{U}$ can open an encapsulation $\mathsf{enc}_{p_e,S'}$ associated to a policy $p_e \in \mathcal{P}_\Omega$ if and only if the conjunction of p_u and p_e is valid with respect to both S and S':

$$(p_u, p_e) \in R \iff (p_u \wedge p_e) \in (\mathcal{V}_S \cap \mathcal{V}_{S'})$$

Remark. It is important to distinguish the access structure in which an encapsulation was created from the one in which a user secret key was created since the universe of expressible postulates can be modified in such a way that an access policy that is valid w.r.t. a first access structure S becomes invalid w.r.t. the new access structure S'.

4.2.1 Examples

1. Without access structure modification

   Let $S = \{A, (H, \iota)\}$, where $A = \{a_1, a_2, a_3\}$ is an anarchical dimension, and $H = \{1_h, 2_h\}$ is a hierarchical dimension with $\iota(2_h) > \iota(1_h)$.

   - Let $p_u = a_1 \wedge 2_h$ and $p_e = a_1 \wedge 2_h$. In this case, a user secret key $\mathsf{usk}_{p_u,\mathcal{U}}$ *can* open an encapsulation $\mathsf{enc}_{p_e,\mathcal{U}}$ since $p_u \wedge p_e = a_1 \wedge 2_h \neq \mathsf{False}$.
   - Let $p_u = a_1 = a_1 \wedge 1_h$ using the definition of a hierarchical dimension. Let $p_e = 2_h = 2_h \wedge \left(\vee_{i=1}^{3} a_i\right) = (2_h \wedge a_1) \vee_{i=2}^{3} (2_h \wedge a_i)$ using the definition of an anarchical dimension.
     In this case, a user secret key $\mathsf{usk}_{p_u,\mathcal{U}}$ *cannot* open an encapsulation $\mathsf{enc}_{p_e,\mathcal{U}}$ since $\neg(1_h \Longrightarrow_\mathcal{U} 2_h)$ by definition of a hierarchical dimension.

2. With expressible universe modification

   (a) Additive transformation

   Let $\mathcal{U}' = \mathcal{U} \cup D_3$, where $D_3 = \{3_1, 3_2\} \subset \mathcal{A}_\Omega$ is an anarchical dimension.

   - Let $p_u = 1_1 \wedge 2_2$ and $p_e = 1_1 \wedge 2_2 \wedge 3_2$. Then a user secret key $\mathsf{usk}_{p_u,\mathcal{U}}$ *cannot* decrypt $\mathsf{enc}_{p_e,\mathcal{U}'}$ since $p_e =_\mathcal{U} \mathsf{False}$ and $\neg(p_u \Longrightarrow_\mathcal{U} \mathsf{False})$.
     $p_u =_{\mathcal{U}'} 1_1 \wedge 2_2 \wedge (3_1 \vee 3_2)$
   - Let $p_u = 1_1 \wedge 2_2$ and $p_e = 1_1 \wedge 2_2 \wedge 3_2$. Then a user secret key $\mathsf{usk}_{p_u,\mathcal{U}}$ *cannot* decrypt $\mathsf{enc}_{p_e,\mathcal{U}'}$ since

   (b) Destructive transformation

4.3 TODO Live universe modifications

Evolving the access structure underlying a given revision of a Covercrypt instance can be done through one of those functions:

add-hierarchy: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-hierarchy: $\mathcal{K}_S \times \mathbb{P}(\mathcal{R}) \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S and a set of attributes $D \subset \mathcal{R}$. If this new hierarchy is valid with respect to S, it returns a new master secret key and a new master public key which structure contains (resp. does not contain) H. Otherwise $\perp$ is returned.

add-anarchy: $\mathcal{K}_S \times \mathcal{A} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-anarchy: $\mathcal{K}_S \times \mathcal{A} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S and an anarchical dimension $A \in \mathcal{A}$. If this new anarchy is valid with respect to S, it returns a new master secret key and a new master public key which structure contains (resp. does not contain) A. Otherwise $\perp$ is returned.

add-attribute: $\mathcal{K}_S \times \mathcal{A} \times \mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-attribute: $\mathcal{K}_S \times \mathcal{A} \times \mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S, an anarchical dimension $D \in \mathcal{A}$ and an attribute $p \in \mathcal{U}$. If this dimension is a member of S, it returns a new master secret key and a new master public key with an access structure in which this dimension contains (resp. does not contain) this attribute.

add-hierarchical-attribute: $\mathcal{K}_S \times \mathcal{H} \times \mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

del-hierarchical-attribute: $\mathcal{K}_S \times \mathcal{H} \times \mathcal{U} \to \mathcal{K}_S \times \mathcal{K}_P$ or $\perp$

This randomized algorithm takes as input a master secret key $msk \in \mathcal{K}_S$ with an access structure S, a hierarchical dimension $D \in \mathcal{H}$ and an attribute $p \in \mathcal{U}$. If this dimension is a member of S, it returns a new master secret key and a new master public key with an access structure in which this dimension contains (resp. does not contain) this attribute.

4.4 TODO Hybridization à la carte

5 TODO Optional key tracing

6 TODO Cooperating Covercrypt authorities

Covercrypt suppose the existence of an central authority in charge of maintaining the master secret key, and generating the master public key and all user secret keys. However, in some use cases, it is desirable to distribute the power over multiple authorities: users managed by a given authority should be able to create encapsulations for attributes defined by a different authority.

Doing so requires (1) generating fragments of a master public key by selecting the subset of the rights from the original key that is targeted for sharing, and (2) combining master public key fragments together to create a new master public key:

```
select: mpk -> policy -> mpk
```

```
combine: mpk -> mpk -> mpk
```

Doing so requires a broader acception of what a master public key is since such key may not contain all rights hold by its associated master secret key, and may not even be associated to a single master secret key anymore.